

Trade Pilot App — 2 Tahap Penyelesaian P0

Pemecahan 4 temuan kritis jadi dua paket kerja yang bisa dirilis terpisah.

Turunan dari Technical Audit Report · commit `8c43038` · branch `feat/flutter-app`

Empat temuan P0 **tidak dipecah berdasarkan besarnya**, tapi berdasarkan **apa yang divalidasi**. Tahap 1 semuanya kode Dart yang menyentuh alur sesi — divalidasi di HP fisik, risiko regresi tinggi, dan harus rilis bareng karena saling bergantung. Tahap 2 semuanya konfigurasi native — divalidasi lewat upload ke store, bukan lewat device.

Kenapa pemisahannya penting: kalau digabung, satu penolakan store bisa menahan perbaikan keamanan lo berminggu-minggu. Dipisah, Tahap 1 bisa jalan duluan tanpa nunggu apa pun.

TAHAP	ISI	SIFAT	DIVALIDASI LEWAT
Tahap 1 rilis 1.0.2	P0-2 backup Android · upgrade secure_storage · P0-4 interceptor 401 · P0-1 hapus password tersimpan	Mendesak — kerentanan aktif di HP user sekarang	Device fisik iOS + Android, termasuk uji jalur upgrade
Tahap 2 gate rilis	P0-3 privacy manifest iOS · regresi di release build · submisi store	Menghambat — bukan bahaya, tapi bisa nahan submission	Upload TestFlight & Play internal testing

Satu rekalibrasi dari report sebelumnya. Gue nulis P0-3 (privacy manifest) sebagai kritis, dan itu perlu diperjelas: lo *sudah* berhasil merilis 1.0.1 ke store. Kalau submission itu lolos tanpa email peringatan `ITMS-91053` dari App Store Connect, berarti manifest bawaan Flutter engine dan plugin-plugin lo sudah cukup — dan P0-3 turun jadi kebersihan, bukan penghalang. **Cek inbox developer lo dulu sebelum mengerjakannya.** Itu sebabnya P0-3 gue taruh di Tahap 2 dan diawali langkah verifikasi, bukan langsung eksekusi.

Amankan sesi & kredensial

Empat langkah berurutan. Urutannya bukan selera — tiap langkah memasang pengaman yang dibutuhkan langkah sesudahnya. Semuanya rilis sebagai satu versi.

Mencakup	P0-2, P0-4, P0-1 + upgrade <code>flutter_secure_storage</code>
File tersentuh	±6 file • 2 titik pemakaian secure storage
Rilis	1.0.2+4

1 Matikan Android Auto Backup

5 MENIT • NOL RISIKO

`android/app/src/main/AndroidManifest.xml`

Kerjakan ini pertama justru karena paling sepele — jangan sampai kelupaan di tengah pekerjaan yang lebih besar. Tambahkan pada tag `<application>` :

```
android:allowBackup="false"
android:fullBackupContent="false"
android:dataExtractionRules="@xml/data_extraction_rules"
```

Buat juga `android/app/src/main/res/xml/data_extraction_rules.xml` dengan `<cloud-backup>` dan `<device-transfer>` kosong (Android 12+ membaca file ini, bukan atribut lama).

Kenapa di posisi 1. Selama ini masih `true`, setiap siklus backup Google menyalin blob kredensial user ke Drive. Begitu di-set `false` dan dirilis, backup lama untuk app ini ikut dihapus di siklus berikutnya — jadi ini yang membersihkan jejak password yang sudah terlanjur ke cloud.

Efek samping yang harus lo terima. User yang pindah HP dan restore dari backup tidak lagi membawa sesi — mereka harus login ulang sekali. Itu memang perilaku yang benar, dan sekaligus menghilangkan bug diam-diam di mana blob terenkripsi ter-restore tanpa kunci Keystore-nya lalu bikin app crash saat startup.

Selesai kalau. Build release Android jalan, dan `adb shell bmgr` tidak lagi mendaftarkan app ini sebagai peserta backup.

2 Upgrade flutter_secure_storage v9.2.4 → v11 FONDASI

pubspec.yaml · lib/core/storage/token_storage.dart:11–12 · lib/screens/auth/login_screen.dart:30–31

Cuma ada **dua titik pemakaian** di seluruh codebase, dan keduanya memakai baris yang sama persis:

`AndroidOptions(encryptedSharedPreferences: true)`. Itu justru flag yang jadi titik migrasi utama — implementasi Android di versi 10+ berubah karena Google mendepresiasi library `androidx.security:security-crypto` yang dipakai versi lama.

Kenapa sebelum langkah 3 dan 4. Langkah 4 menulis ulang seluruh alur penyimpanan kredensial. Kalau lo tulis dulu dengan API v9 lalu upgrade, lo menulis kode yang sama dua kali dan menguji dua kali di device. Upgrade dulu, baru bangun alur baru di atas API final.

Risiko yang wajib diuji, bukan diasumsikan. Kalau backend penyimpanan Android berubah, token yang sudah tersimpan di HP user bisa jadi tidak terbaca lagi — artinya **semua user terpaksa login ulang sekali**. Baca changelog v10 dan v11 sebelum mulai, lalu uji langsung: install build lama, login, lalu upgrade in-place ke build baru dan lihat apakah sesinya bertahan.

Kalau ternyata memang memaksa login ulang — itu justru alasan kuat untuk tetap menggabungkannya di Tahap 1 ini: user cukup sekali dipaksa login ulang, bareng dengan pembersihan password di langkah 4, bukan dua kali di dua rilis berbeda.

Selesai kalau. flutter analyze bersih, 151 test masih lulus, dan read/write token terbukti jalan di device fisik Android dan iOS — bukan cuma simulator.

3 Pasang interceptor 401 terpusat (P0-4)

JARING PENGAMAN

packages/trade_pilot_api_client/lib/trade_pilot_client.dart · lib/providers/auth_provider.dart · lib/main.dart

Tambahkan `onError` interceptor di `TradePilotClient` — seam-nya sudah lo siapkan sendiri lewat getter `Dio` `get dio` dan komentarnya. Saat `statusCode == 401`, panggil callback `onUnauthorized` yang di-inject `AuthProvider`; callback itu membersihkan token lokal, set status `unauthenticated`, lalu `notifyListeners()`.

Kenapa harus sebelum langkah 4. Ini yang paling penting dari seluruh rencana. Sekarang, kalau token user mati di tengah sesi, mereka terjebak — tapi setidaknya password mereka masih terisi di form, jadi ada jalan keluar. Langkah 4 mencabut jalan keluar itu. **Kalau lo kerjakan langkah 4 duluan, lo membuat jebakannya jadi lebih parah, bukan lebih baik.** Pasang pintu daruratnya dulu, baru cabut tongkatnya.

Dua jebakan implementasi.

Satu — kecualikan endpoint auth (`/auth/login`, `/auth/register`, `/auth/forgot-password/*`) dari interceptor. Kalau tidak, user yang salah ketik password akan dapat 401 lalu dilempar sebagai "sesi berakhir" padahal mereka memang belum login. Sisakan juga penanganan 401 lokal di `changePassword` dan `deleteAccount`, karena di sana 401 memang berarti "password saat ini salah", bukan sesi mati.

Dua — `SplashScreen` memang otomatis merutekan ulang karena dia `Consumer<AuthProvider>` yang membaca `AuthStatus` — tapi dia dipasang sebagai `home`: di `MaterialApp`. Route yang di-*push* di atasnya (`AnalysisDetailScreen`, `TradeJournalScreen`, dan 25 `Navigator.push` lainnya) **tidak ikut ter-pop** — jadi user akan lihat `LoginScreen` ter-render di bawah layar detail yang masih menutupi. Tambahkan `navigatorKey` di `MaterialApp` dan panggil `popUntil((r) => r.isFirst)` saat forced logout.

Selesai kalau. Lo invalidasi token dari sisi server, buka `AnalysisDetailScreen`, tarik-refresh — dan app kembali ke layar login dengan stack bersih, bukan menumpuk di belakang.

4 Hapus penyimpanan password, ganti token bergerbang biometrik (P0-1) INTI

lib/screens/auth/login_screen.dart:24–25, 63–70, 74–88, 111–126 · lib/core/storage/token_storage.dart · test/screens/auth/login_screen_test.dart

Alur barunya:

- Login sukses → token sudah tersimpan di `TokenStorage`. Simpan flag `biometric_enabled = true` dan email saja (email tidak sensitif, tetap boleh untuk prefill).
- Buka app → kalau flag ada dan biometrik tersedia, tampilkan tombol biometrik. Setelah `authenticate()` berhasil, **baca token yang sudah ada** lalu panggil `getMe()` untuk memulihkan sesi. Tidak ada password yang terlibat sama sekali.
- Hapus `_rememberedPasswordKey` beserta seluruh pemakaiannya.
- Tambahkan **migrasi sekali-jalan**: di startup versi baru, `delete` key `remembered_login_password` tanpa syarat, ditandai flag di `SharedPreferences` supaya cuma jalan sekali. Ini yang membersihkan password di HP user yang sudah terlanjur menyimpannya.

Test yang akan gagal — dan itu memang benar. `test/screens/auth/login_screen_test.dart:51` punya test bernama *"login restores remembered credentials from secure storage"* yang saat ini **meng-assert perilaku yang rentan itu** — dia memverifikasi bahwa password ter-restore dari storage. Jangan dihapus: **balik assertion-nya** jadi memastikan field password tetap kosong dan key `remembered_login_password` tidak pernah dibaca. Test itu berubah dari pengunci bug jadi penjaga perbaikan.

Selesai kalau. Uji jalur upgrade di device fisik: install build lama → login dengan "Remember me" aktif → upgrade in-place ke build baru → password hilang dari form, key lama terhapus dari storage, dan tombol biometrik tetap membuka sesi tanpa mengetik apa pun.

Definition of done — Tahap 1

- ☐ `flutter analyze` bersih, seluruh test lulus (login test sudah dibalik)
- ☐ Uji jalur upgrade dari build lama di HP Android fisik *dan* iPhone fisik
- ☐ Verifikasi key `remembered_login_password` benar-benar hilang dari storage pasca-migrasi
- ☐ Uji forced logout dari dalam route yang di-push — stack bersih, tidak menumpuk
- ☐ Uji salah password di layar login tidak memicu pesan "sesi berakhir"
- ☐ Uji biometrik: aktif, gagal, dan dibatalkan user — ketiganya tidak bikin app buntu

Lolos gerbang store

Konfigurasi native dan submisi. Tidak menyentuh logika app sama sekali, jadi risiko regresinya beda jenis — yang diuji di sini bukan perilaku di device, tapi apakah build-nya diterima store.

Mencakup **P0-3 + regresi release build + submisi**

Alat **Xcode, App Store Connect, Play Console**

Prasyarat **Tahap 1 sudah merge dan lulus uji device**

5 Verifikasi dulu: apakah P0-3 benar-benar menghalangi?

CEK SEBELUM KERJA

App Store Connect · inbox developer

Sebelum menulis satu baris pun, jawab satu pertanyaan: **waktu lo submit 1.0.1, apakah ada email ITMS-91053: Missing API declaration dari App Store Connect?**

- **Tidak ada email** → manifest bawaan Flutter engine dan plugin sudah menutupi kebutuhan lo. P0-3 turun jadi kebersihan, bukan penghalang. Kerjakan langkah 6 sebagai pelengkap, tapi jangan tahan rilis karenanya.
- **Ada email** → email itu menyebutkan API mana persisnya yang belum dideklarasikan. Itu jadi daftar isian langkah 6 lo, dan langkah 6 wajib selesai sebelum submit berikutnya.

Kenapa ini jadi langkah tersendiri. Upgrade `flutter_secure_storage` ke v11 di Tahap 1 sudah membawa privacy manifest milik plugin itu sendiri — yang di v9 memang belum ada. Jadi ada kemungkinan nyata Tahap 1 sudah menyelesaikan sebagian besar masalah ini tanpa lo sengaja. Cek dulu, jangan kerjakan yang mungkin sudah beres.

6 Buat PrivacyInfo.xcprivacy (P0-3)

`ios/Runner/PrivacyInfo.xcprivacy` — belum ada

Buat file-nya, lalu **tambahkan ke target Runner lewat Xcode** (bukan cuma taruh di folder — kalau tidak terdaftar di build phase, dia tidak ikut ter-bundle dan lo akan mengira sudah beres padahal belum). Isinya:

- `NSPrivacyTracking` → `false`
- `NSPrivacyTrackingDomains` → array kosong
- `NSPrivacyAccessedAPITypes` → `UserDefaults` dengan reason `CA92.1` (dipakai `shared_preferences` untuk tema, bahasa, dan preferensi filter history)
- `NSPrivacyCollectedDataTypes` → email dan user ID, purpose *App Functionality*, tertaut ke identitas, **tidak** dipakai untuk tracking

Jaga konsistensi dengan yang lo isi di store. Deklarasi di file ini harus cocok dengan jawaban App Privacy di App Store Connect dan Data Safety di Play Console. Ketidacocokan antara keduanya justru jenis temuan yang bikin review manual berlarut — lebih lama daripada kalau manifest-nya tidak ada sama sekali.

Selesai kalau. Upload build ke TestFlight dan **tidak** menerima email peringatan dalam 24 jam.

7 Regresi Tahap 1 di release build, bukan debug GERBANG

`build release Android (signed) + iOS (archive)`

Semua pengujian di Tahap 1 dilakukan di build debug atau profile. Ulangi **seluruh definition of done Tahap 1** sekali lagi di build release yang sudah ditandatangani. Ini bukan formalitas: Keystore Android dan Keychain iOS berperilaku berbeda antara build debug dan release — debug memakai signing key yang berbeda, dan itu memengaruhi entri Keystore yang jadi tempat `flutter_secure_storage` menyimpan kuncinya.

Ini titik paling mungkin gagal di seluruh rencana. Alur kredensial yang lulus sempurna di debug bisa gagal membaca token di release. Jangan jadwalkan langkah ini di hari yang sama dengan rencana submit — sisakan jarak.

Selesai kalau. Login, biometrik, forced logout, dan migrasi penghapusan password semuanya terbukti jalan di AAB dan IPA hasil build release.

8 Submisi

Play Console `internal testing` → `production` · TestFlight → App Review

Naikkan versi ke `1.0.2+4`. Lepas ke jalur internal testing dulu di kedua store, tahan minimal 24 jam dengan beberapa penguji nyata yang **meng-upgrade dari 1.0.1** — bukan install bersih. Jalur upgrade itu satu-satunya yang bisa membuktikan migrasi penghapusan password beneran jalan di kondisi asli.

Selesai kalau. Naik ke production di kedua store, dan Sentry (kalau P1-5 sudah dipasang) tidak menunjukkan lonjakan error auth dalam 48 jam pertama.

Ringkasan urutan

LANGKAH	PEKERJAAN	TAHAP	BERGANTUNG PADA
1	<code>allowBackup="false"</code>	Tahap 1	—
2	Upgrade <code>secure_storage</code> v11	Tahap 1	—
3	Interceptor 401 terpusat	Tahap 1	—
4	Hapus password, biometrik pakai token	Tahap 1	Langkah 2 dan 3
5	Verifikasi perlu-tidaknya P0-3	Tahap 2	Langkah 2
6	<code>PrivacyInfo.xcprivacy</code>	Tahap 2	Langkah 5
7	Regresi di release build	Tahap 2	Seluruh Tahap 1
8	Submisi ke store	Tahap 2	Langkah 6 dan 7

Kalau lo cuma sempat mengerjakan satu hal minggu ini: langkah 3 (interceptor 401). Dia berdiri sendiri tanpa bergantung apa pun, memperbaiki jebakan yang user lo mungkin sudah alami hari ini, dan dia adalah prasyarat yang bikin langkah 4 aman dikerjakan. Langkah 1 nyusul — lima menit.